Automated Test Case Execution & Reporting

Hệ thống Thương mại Điện tử EduCommerce Checkout System

Học phần: Kiểm thử Phần mềm (Software Testing)
Sinh viên thực hiện: Lê Thị Ngọc Anh
Mã số sinh viên: 23110131
Giảng viên hướng dẫn: TS. Trần Tuấn Anh
GitHub Repository: `https://github.com/ngocanh1002/STLab02_EduCommerce_Testing`

Mục lục

Part 1 - Phân tích Requirement & Thiết kế Bộ kiểm thử Tự động **2**
1.1. Tổng quan Ứng dụng EduCommerce & 5 Requirements Lựa chọn . . . 2
1.2. Ma trận Test Condition & Test Scenario Matrix . . . 3
1.3. Bảng Đặc tả Chi tiết 19 Test Cases (Chuẩn Schema 15 cột) . . . 4
Part 2 - Bằng chứng Thực thi, Giải thích Mã nguồn & Báo cáo Kiểm thử **6**
2.1. Kiến trúc Kiểm thử Tự động & Giải thích Mã nguồn Trọng yếu . . . 6
2.2. Danh mục Bằng chứng Thực thi (Test Execution Evidences E01 - E10) . . . 7
2.3. Báo cáo Tổng hợp Kết quả Thực thi (Execution Summary) . . . 16
2.4. Báo cáo Lỗi DEF-PAY-01 & Phân tích Nguyên nhân Gốc rễ (Root Cause Analysis) . . . 16
Part 3 - Giải quyết Tình huống Thực tế: Problem Solving **18**
3.1. Hai SKU, Một SKU Thiếu Tồn Kho . . . 18
3.2. Voucher Chỉ Đánh Dấu Used Sau Khi Thanh Toán Thành Công . . . 18
3.3. Mã Phương thức Thanh toán Lạ . . . 19
3.4. Giá Sản phẩm Thay đổi Giữa Add-to-cart và Checkout . . . 19
3.5. Cùng SKU Nhập Hai Lần Vào Giỏ Hàng . . . 20
3.6. Data-driven Tests Bị Phụ Thuộc Nhau (Shared State Pollution) . . . 20
3.7. Dashboard Báo PASS Dù Thiếu Test Do Parse Lỗi . . . 21
3.8. Đổi Quy định Miễn Phí Vận Chuyển (Change Impact Analysis) . . . 21
Part 4 - Trả lời Câu hỏi Phỏng vấn Chuyên môn & Code Review (15%) **22**
4.1. Lời giải 20 Câu hỏi Phỏng vấn Lý thuyết Kiểm thử (Mục 4) . . . 22
4.2. Lời giải 03 Bài toán Code Review Chuyên sâu (Mục 4.1 trong Đề bài) . . . 26
Part 5 - Tự Đánh giá, Giới hạn Phạm vi & Thông tin Nộp bài **28**

Part 1 - Phân tích Requirement & Thiết kế Bộ kiểm thử Tự động

1.1. Tổng quan Ứng dụng EduCommerce & 5 Requirements Lựa chọn

Ứng dụng **EduCommerce** là hệ thống đặt hàng trực tuyến thu nhỏ mô phỏng chu trình người mua: Catalog → Cart → Promotions + Shipping → Inventory → Payments → Checkout → Order.

Yêu cầu: Không dùng lại 3 requirement đã giải mẫu gồm REQ-06: Mã giảm giá hợp lệ, REQ-08: Giảm giá tối đa, REQ-11: Idempotent checkout. Báo cáo lựa chọn 5 REQs: 02, 04, 07, 09, 10.

Req ID	Module	Requirement / Acceptance Rules	Actor	Question / Assumption	Risk
REQ-02	Cart	• Số lượng thêm vào giỏ phải là số nguyên dương ($quantity > 0$). • Khi thêm cùng một SKU nhiều lần, hệ thống phải cộng dồn số lượng vào cùng một dòng sản phẩm (*CartLine*), không tạo dòng mới trùng lặp.	Customer / Cart	• Có giới hạn số lượng tối đa cho mỗi SKU trên một đơn hàng không? (**Assumption**: Giới hạn bởi tồn kho hiện có trong Catalog). • Nếu thêm số lượng 0 hoặc số âm thì hệ thống bỏ qua hay ném ngoại lệ? (**Assumption**: Báo ngoại lệ `ValueError('INVALID_QUANTITY')`)	• Tạo nhiều dòng trùng SKU làm sai lệch logic trừ kho. • Số lượng âm làm giảm tổng tiền hoặc hack hệ thống hoàn tiền ảo.
REQ-04	Authentication	• Chỉ khách hàng đã đăng nhập thành công (`customer.authenticated == True`) mới được phép tạo đơn hàng tại Checkout. • Khách hàng vãng lai (`authenticated == False`) bị từ chối ngay lập tức với lỗi `LOGIN_REQUIRED`.	Customer / Checkout	• Có hỗ trợ Guest Checkout không? (**Assumption**: Theo tài liệu nghiệp vụ EduCommerce, không hỗ trợ Guest Checkout, bắt buộc đăng nhập). • Lỗi login có làm giữ lại giỏ hàng không? (**Assumption**: Giỏ hàng được bảo lưu trong session).	• Người dùng chưa xác thực tạo được đơn → Không xác định được chủ sở hữu đơn hàng để giao và khiếu nại.
REQ-07	Promotions	• Mã voucher không tồn tại hoặc đã nằm trong danh sách đã sử dụng của người dùng (`customer.used_codes`) phải bị từ chối. • Mã voucher chỉ được chính thức đưa vào `used_codes` khi đơn hàng được tạo và thanh toán thành công.	Customer / Promotions / Checkout	• Nếu thanh toán Online bị cổng từ chối (Declined), mã voucher có bị đánh dấu là đã dùng không? (**Assumption**: Không được đánh dấu đã dùng, phải bảo toàn voucher cho khách mua lại).	• Khách mất voucher oan khi thanh toán lỗi mạng. • Tái sử dụng voucher nhiều lần làm thất thoát lớn ngân sách khuyến mãi.

Req ID	Module	Requirement / Acceptance Rules	Actor	Question / Assumption	Risk
REQ-09	Shipping	• Giao hàng nội thành miễn phí cước vận chuyển khi tổng tiền hàng subtotal $\geq$ 500.000đ; nếu subtotal $<$ 500.000đ thì tính phí cố định 30.000đ. • Giao hàng ngoại thành áp dụng phí 50.000đ không phụ thuộc vào giá trị subtotal. Khu vực không hợp lệ báo lỗi `INVALID_AREA`.	Shipping / Checkout	• Ngưỡng 500.000đ xét trên Subtotal tiền hàng trước discount hay Total sau discount? (**Assumption**: Xét trên `subtotal` theo đặc tả module Shipping). • Biên chính xác là $\geq$ 500.000đ hay $>$ 500.000đ? (**Assumption**: Đúng 500.000đ được freeship).	• Lỗi tính sai phí ship gây rối loạn khách hàng hoặc công ty phải chịu bù lỗ cước bưu điện. • Nhầm lẫn toán tử $<$ và $\leq$ tại ngưỡng 500.000đ.
REQ-10	Payments	• Phương thức COD tạo đơn hàng ở trạng thái `PENDING_COD`. • Phương thức ONLINE gửi yêu cầu sang cổng thanh toán; nếu được chấp thuận (`approved == True`) thì đơn hàng chuyển sang `PAID` và ghi nhận bản ghi giao dịch; nếu bị từ chối (`approved == False`) thì ném lỗi `PAYMENT_DECLINED` và không tạo đơn hàng.	Payment Gateway / Checkout	• Cổng thanh toán bên thứ ba giả lập qua cờ `approved` boolean. • Khi payment declined, tồn kho đã reserve có được hoàn trả không? (**Assumption**: Không được lưu vết đơn hàng trong `orders`).	• Tạo đơn hàng giả mạo dù chưa thanh toán. • Trừ tiền khách hàng nhưng không ghi nhận trạng thái PAID.

1.2. Ma trận Test Condition & Test Scenario Matrix

Ma trận dưới đây thể hiện sự chuyển hóa từ 5 Requirements sang 12 Test Conditions và 12 Test Scenarios:

Req ID	Condition ID	Scenario ID	Test Scenario Description	Testing Level	Testing Type	Testing Approach
REQ-02	COND-CRT-01	TS-CRT-VALID	Thêm sản phẩm hợp lệ ($qty > 0$) và cộng dồn số lượng khi thêm trùng SKU nhiều lần	Unit / Component	Functional	Equivalence Partitioning
REQ-02	COND-CRT-02	TS-CRT-INVALID	Thêm sản phẩm với số lượng không hợp lệ ($qty = 0$ hoặc $qty < 0$)	Unit / Component	Negative	Boundary Value Analysis
REQ-04	COND-AUT-01	TS-AUT-LOGIN	Khách hàng đã đăng nhập xác thực checkout thành công	System / E2E	Functional	Positive / Happy Path
REQ-04	COND-AUT-02	TS-AUT-GUEST	Khách hàng chưa đăng nhập bị chặn checkout và không sinh đơn hàng	System / Integration	Security / Functional	Negative Testing
REQ-07	COND-PRM-01	TS-PRM-INVALID	Áp dụng mã voucher không tồn tại trong danh mục hệ thống	Integration	Functional	Negative Testing

Req ID	Condition ID	Scenario ID	Test Scenario Description	Testing Level	Testing Type	Testing Approach
REQ-07	COND-PRM-02	TS-PRM-REUSE	Áp dụng mã voucher đã nằm trong lịch sử used_codes của người dùng	Integration	Functional	Negative / Anti-abuse
REQ-07	COND-PRM-03	TS-PRM-SUCCESS	Đơn hàng thanh toán thành công cập nhật mã voucher vào used_codes	Integration	State Transition	Positive Testing
REQ-07	COND-PRM-03	TS-PRM-ROLLBACK	Đơn hàng thanh toán online thất bại không được ghi nhận mã vào used_codes	Integration	State Transition	Negative / Exception
REQ-09	COND-SHP-01	TS-SHP-INNER	Tính phí ship nội thành tại các biên: đúng 500k, dưới 500k (499.999đ), trên 500k	Unit / Component	Functional	3-point Boundary Value
REQ-09	COND-SHP-02	TS-SHP-OUTER	Tính phí ship ngoại thành cố định 50.000đ không phụ thuộc subtotal	Unit / Component	Functional	Equivalence Partitioning
REQ-09	COND-SHP-03	TS-SHP-INVALID	Khu vực vận chuyển không hợp lệ ném ngoại lệ INVALID_AREA	Unit / Component	Robustness	Negative Testing
REQ-10	COND-PAY-01	TS-PAY-COD	Thanh toán COD thành công tạo Order trạng thái PENDING_COD	Integration	Functional	State Verification
REQ-10	COND-PAY-02	TS-PAY-ONLINE-OK	Thanh toán ONLINE thành công (approved) tạo Order trạng thái PAID và lưu Charge	Integration	Functional	State Verification
REQ-10	COND-PAY-03	TS-PAY-ONLINE-FAIL	Thanh toán ONLINE bị declined ném lỗi PAYMENT_DECLINED và không tạo Order	Integration	Robustness	Negative / State Guard

1.3. Bảng Đặc tả Chi tiết 19 Test Cases

Bộ dữ liệu kiểm thử với 19 test cases lưu trong `STLab02_TestCases.csv`. Bao gồm **9 Positive Cases** và **10 Negative/Boundary Cases**:

Test Case ID	Req	Scenario	Title / Mục tiêu kiểm thử	Prior	Precondition / Fixture	Input Data (JSON)	Expected Output (JSON)
TC-CRT-001	REQ-02	TS-CRT-VALID	Thêm sản phẩm mới hợp lệ vào giỏ hàng	P1	Cart rỗng, SKU A giá 200k / `clean_cart`	`{"actions": [{"sku": "A", "quantity": 2}]}`	`{"total_quantity": 2, "lines_count": 1, "subtotal": 400000}`
TC-CRT-002	REQ-02	TS-CRT-VALID	Cộng dồn số lượng khi thêm cùng SKU nhiều lần	P1	Cart đã có SKU A qty=2, thêm A qty=3 / `clean_cart`	`{"actions": [{"sku": "A", "quantity": 2}, {"sku": "A", "quantity": 3}]}`	`{"total_quantity": 5, "lines_count": 1, "subtotal": 1000000}`
TC-CRT-003	REQ-02	TS-CRT-INVALID	Thêm sản phẩm số lượng bằng 0 (Biên không hợp lệ)	P0	Cart rỗng, SKU A active / `clean_cart`	`{"actions": [{"sku": "A", "quantity": 0}]}`	`{"error": "INVALID_QUANTITY"}`

Test Case ID	Req	Scenario	Title / Mục tiêu kiểm thử	Prior.	Precondition / Fixture	Input Data (JSON)	Expected Output (JSON)
TC-CRT-004	REQ-02	TS-CRT-INVALID	Thêm sản phẩm với số lượng âm (Negative test)	P1	Cart rỗng, SKU A active / clean_cart	{"actions": [{"sku": "A", "quantity": -1}]}	{"error": "INVALID_QUANTITY"}
TC-AUT-001	REQ-04	TS-AUT-LOGIN	Khách hàng đã đăng nhập checkout COD thành công	P1	Customer authenticated=True, giỏ 600k / 600k	{"method": "COD"}	{"status": "PENDING_COD"}
TC-AUT-002	REQ-04	TS-AUT-GUEST	Khách hàng chưa đăng nhập bị từ chối checkout (Negative)	P0	Customer authenticated=False / unauth_user	{"method": "COD"}	{"error": "LOGIN_REQUIRED"}
TC-AUT-003	REQ-04	TS-AUT-GUEST	Khách chưa đăng nhập không tạo Order trong hệ thống (Negative)	P1	Customer unauthenticated, Online / unauth_user	{"method": "ONLINE", "approved": true}	{"error": "LOGIN_REQUIRED", "orders_count": 0}
TC-PRM-001	REQ-07	TS-PRM-INVALID	Từ chối voucher không tồn tại trong hệ thống (Negative)	P1	Customer authenticated, giỏ 600k / 600k	{"voucher": "INVALID_CODE_999"}	{"error": "INVALID_VOUCHER"}
TC-PRM-002	REQ-07	TS-PRM-REUSE	Từ chối voucher đã nằm trong used_codes của user (Negative)	P0	User đã dùng SAVE10 / used_voucher	{"voucher": "SAVE10"}	{"error": "VOUCHER_USED"}
TC-PRM-003	REQ-07	TS-PRM-SUCCESS	Đánh dấu voucher vào used_codes khi order thành công	P0	User mới, SAVE10, giỏ 600k COD / 600k	{"voucher": "SAVE10", "method": "COD"}	{"voucher_in_used_codes": true, "discount": 60000}
TC-PRM-004	REQ-07	TS-PRM-ROLLBACK	Không đánh dấu voucher used khi Online bị Declined (Negative)	P0	User mới, SAVE10, Online declined / 600k	{"voucher": "SAVE10", "method": "ONLINE", "approved": false}	{"error": "PAYMENT_DECLINED", "voucher_in_used_codes": false}
TC-SHP-001	REQ-09	TS-SHP-INNER	Miễn phí ship nội thành đúng tại ngưỡng 500.000đ (Biên)	P0	Subtotal đúng 500.000đ, area=inner / 500k	{"area": "inner"}	{"shipping": 0}
TC-SHP-002	REQ-09	TS-SHP-INNER	Phí ship nội thành 30k ngay dưới ngưỡng 499.999đ (Biên)	P0	Subtotal 499.999đ, area=inner / 499999	{"area": "inner"}	{"shipping": 30000}
TC-SHP-003	REQ-09	TS-SHP-INNER	Miễn phí ship nội thành khi subtotal trên ngưỡng 600k	P1	Subtotal 600.000đ, area=inner / 600k	{"area": "inner"}	{"shipping": 0}
TC-SHP-004	REQ-09	TS-SHP-OUTER	Phí ship ngoại thành cố định 50k dù subtotal $\geq$ 500k	P1	Subtotal 600.000đ, area=outer / 600k	{"area": "outer"}	{"shipping": 50000}
TC-SHP-005	REQ-09	TS-SHP-INVALID	Từ chối khu vực vận chuyển không hợp lệ (Negative)	P2	Subtotal 600.000đ, khu vực lạ / 600k	{"area": "hanoi_suburb"}	{"error": "INVALID_AREA"}
TC-PAY-001	REQ-10	TS-PAY-COD	Thanh toán COD thành công tạo Order trạng thái PENDING_COD	P1	User authenticated, giỏ 600k COD / 600k	{"method": "COD"}	{"order_status": "PENDING_COD", "orders_count": 1}
TC-PAY-002	REQ-10	TS-PAY-ONLINE-OK	Thanh toán ONLINE approved tạo Order PAID & lưu charge	P0	User auth, ONLINE approved=True / 600k	{"method": "ONLINE", "approved": true}	{"order_status": "PAID", "charges_count": 1}
TC-PAY-003	REQ-10	TS-PAY-ONLINE-FAIL	Thanh toán ONLINE declined: từ chối, không tạo Order & hoàn kho (Negative/Defect)	P0	User auth, ONLINE approved=False / 600k	{"method": "ONLINE", "approved": false}	{"error": "PAYMENT_DECLINED", "orders_count": 0, "stock_restored": true}

Part 2 - Bằng chứng Thực thi, Giải thích Mã nguồn & Báo cáo kiểm thử

2.1. Kiến trúc Kiểm thử Tự động & Giải thích Mã nguồn trọng yếu

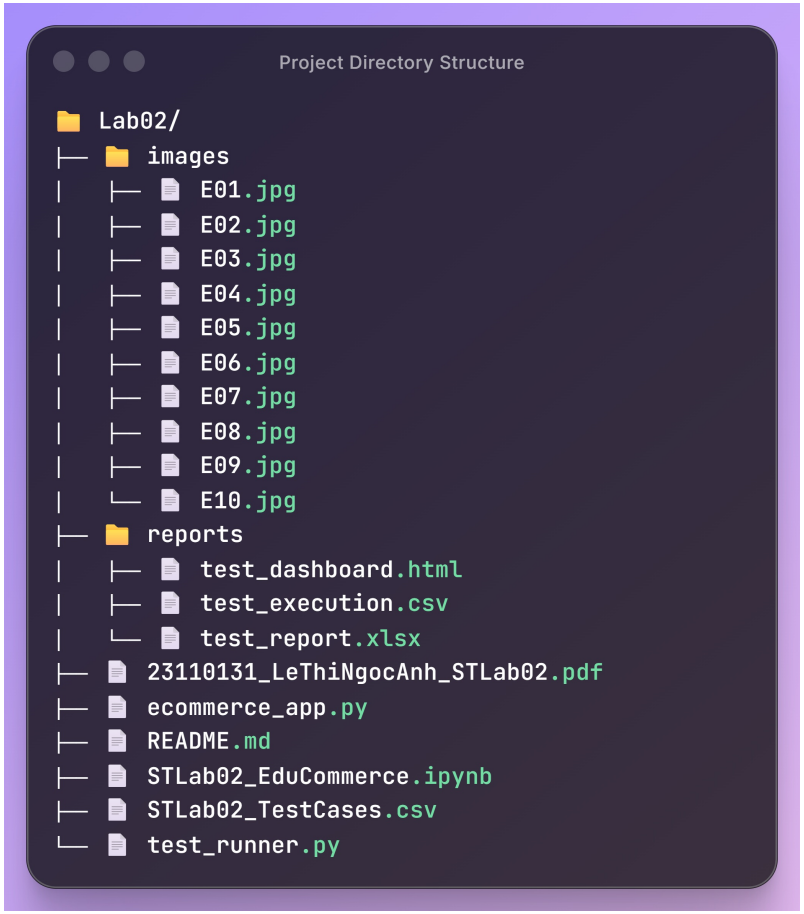
Hệ thống kiểm thử tự động của bài lab được xây dựng trong module `test_runner.py` theo mô hình Data-driven Testing kết hợp Test Dispatcher Architecture gồm 5 thành phần cốt lõi:

1. **Cơ chế Test Isolation thông qua `fixture(subtotal_case)`:** Khởi tạo một đối tượng `Checkout`, `Customer`, và `Cart` hoàn toàn độc lập cho từng dòng test case. Điều này loại bỏ hoàn toàn nguy cơ ô nhiễm dữ liệu chéo (Dirty State) giữa các ca kiểm thử. Các nhánh fixture chính gồm:
   - `clean_cart`: Khởi tạo Cart hoàn toàn mới để kiểm tra các thao tác thêm dòng giỏ hàng của REQ-02.
   - `unauth_user`: Khách hàng có `authenticated = False` phục vụ kiểm thử REQ-04.
   - `used_voucher`: Khách hàng đã có sẵn mã `SAVE10` trong tập hợp `used_codes` phục vụ kiểm thử REQ-07.
   - `499999`: Điều chỉnh giá sản phẩm B thành 299.999đ để tạo subtotal đạt chính xác 499.999đ cho kiểm thử biên REQ-09.
   - `500k`, `600k`: Các giá trị subtotal phân hoạch tương đương chuẩn.
2. **Bộ điều phối Test Dispatcher (`execute_case(row)`):** Tiếp nhận từng dòng dữ liệu từ CSV, đọc cờ `requirement_id` để gọi chính xác phương thức nghiệp vụ tương ứng:
   - REQ-02 → Gọi vòng lặp `cart.add(sku, quantity)` và tổng hợp `total_quantity`, `lines_count`, `subtotal`.
   - REQ-04 → Gọi `app.place(user, cart)` và kiểm tra điều kiện đăng nhập.
   - REQ-07 → Gọi `app.place()` kèm mã voucher và kiểm tra cờ `voucher_in_used_codes`.
   - REQ-09 → Gọi trực tiếp `app.shipping.calculate(subtotal, area)` để kiểm định logic tính cước.
   - REQ-10 → Gọi `app.place()` với các phương thức COD/ONLINE và kiểm tra trạng thái đơn hàng.
3. **Kiểm định giá trị trả về kết hợp Trạng thái Hệ thống (Asserting Return Value vs System State):** Không chỉ kiểm tra giá trị trả về của hàm, runner còn kiểm tra các Side-effects quan trọng: số lượng đơn hàng trong `app.orders`, số lượng giao dịch trong `app.payments.charges`, và trạng thái tồn kho `stock_before` với `stock_after`.
4. **Xử lý Ngoại lệ Kỳ vọng (Expected Exception Handling):** Khối `try...except Exception as e` bắt tất cả ngoại lệ thực tế. Nếu test case kỳ vọng ném lỗi (`'error' in exp`), runner kiểm tra xem chuỗi thông báo ngoại lệ thực tế có khớp tuyệt đối với ngoại lệ yêu cầu hay không. Nếu trùng khớp, kết luận là **PASS** còn nếu phát sinh ngoại lệ không mong muốn, kết luận là **FAIL**.
5. **Hệ thống Báo cáo đa định dạng (Multi-format Reporter):** Runner xuất dữ liệu kết quả ra 3 định dạng: CSV (`test_execution.csv`), Excel 3 sheets (`test_report.xlsx`) và HTML Dashboard độc lập (`test_dashboard.html`).

2.2. Danh mục Bằng chứng Thực thi (Test Execution Evidences E01 – E10)

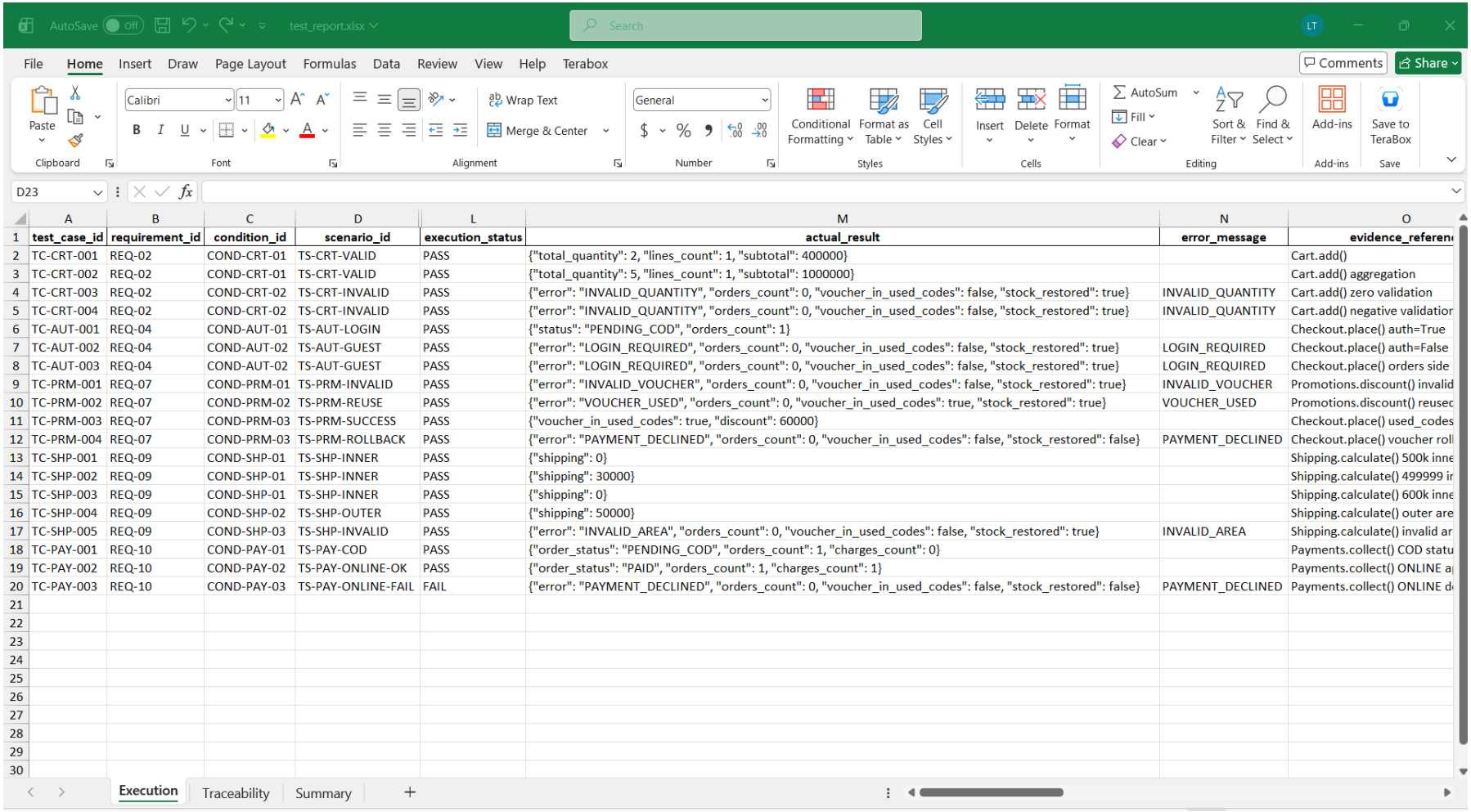
1. Bằng chứng E01 - Cấu trúc Thư mục Dự án (Project Directory Structure):

Thể hiện đầy đủ mã nguồn ứng dụng (`ecommerce_app.py`), bộ test runner (`test_runner.py`), file test case (`STLab02_TestCases.csv`), Jupyter notebook (`STLab02_EduCommerce.ipynb`) và thư mục báo cáo (`reports/`).



2. Bằng chứng E02 - Ma trận Traceability (Traceability Matrix):

Thể hiện mối liên kết xuyên suốt từ Requirement → Test Condition → Test Scenario → Test Case → Execution Status (`text_report.xlxs` mục Execution)



test_report.xlsx

test_case_id	requirement_id	condition_id	scenario_id	execution_status	actual_result	error_message	evidence_referen
TC-CRT-001	REQ-02	COND-CRT-01	TS-CRT-VALID	PASS	{"total_quantity": 2, "lines_count": 1, "subtotal": 400000}		Cart.add()
TC-CRT-002	REQ-02	COND-CRT-01	TS-CRT-VALID	PASS	{"total_quantity": 5, "lines_count": 1, "subtotal": 1000000}		Cart.add() aggregation
TC-CRT-003	REQ-02	COND-CRT-02	TS-CRT-INVALID	PASS	{"error": "INVALID_QUANTITY", "orders_count": 0, "voucher_in_used_codes": false, "stock_restored": true}	INVALID_QUANTITY	Cart.add() zero validation
TC-CRT-004	REQ-02	COND-CRT-02	TS-CRT-INVALID	PASS	{"error": "INVALID_QUANTITY", "orders_count": 0, "voucher_in_used_codes": false, "stock_restored": true}	INVALID_QUANTITY	Cart.add() negative validation
TC-AUT-001	REQ-04	COND-AUT-01	TS-AUT-LOGIN	PASS	{"status": "PENDING_COD", "orders_count": 1}		Checkout.place() auth=True
TC-AUT-002	REQ-04	COND-AUT-02	TS-AUT-GUEST	PASS	{"error": "LOGIN_REQUIRED", "orders_count": 0, "voucher_in_used_codes": false, "stock_restored": true}	LOGIN_REQUIRED	Checkout.place() auth=False
TC-AUT-003	REQ-04	COND-AUT-02	TS-AUT-GUEST	PASS	{"error": "LOGIN_REQUIRED", "orders_count": 0, "voucher_in_used_codes": false, "stock_restored": true}	LOGIN_REQUIRED	Checkout.place() orders side
TC-PRM-001	REQ-07	COND-PRM-01	TS-PRM-INVALID	PASS	{"error": "INVALID_VOUCHER", "orders_count": 0, "voucher_in_used_codes": false, "stock_restored": true}	INVALID_VOUCHER	Promotions.discount() invalid
TC-PRM-002	REQ-07	COND-PRM-02	TS-PRM-REUSE	PASS	{"error": "VOUCHER_USED", "orders_count": 0, "voucher_in_used_codes": true, "stock_restored": true}	VOUCHER_USED	Promotions.discount() reusec
TC-PRM-003	REQ-07	COND-PRM-03	TS-PRM-SUCCESS	PASS	{"voucher_in_used_codes": true, "discount": 60000}		Checkout.place() used_codes
TC-PRM-004	REQ-07	COND-PRM-03	TS-PRM-ROLLBACK	PASS	{"error": "PAYMENT_DECLINED", "orders_count": 0, "voucher_in_used_codes": false, "stock_restored": false}	PAYMENT_DECLINED	Checkout.place() voucher roll
TC-SHP-001	REQ-09	COND-SHP-01	TS-SHP-INNER	PASS	{"shipping": 0}		Shipping.calculate() 500k inne
TC-SHP-002	REQ-09	COND-SHP-01	TS-SHP-INNER	PASS	{"shipping": 30000}		Shipping.calculate() 499999 ir
TC-SHP-003	REQ-09	COND-SHP-01	TS-SHP-INNER	PASS	{"shipping": 0}		Shipping.calculate() 600k inne
TC-SHP-004	REQ-09	COND-SHP-02	TS-SHP-OUTER	PASS	{"shipping": 50000}		Shipping.calculate() outer are
TC-SHP-005	REQ-09	COND-SHP-03	TS-SHP-INVALID	PASS	{"error": "INVALID_AREA", "orders_count": 0, "voucher_in_used_codes": false, "stock_restored": true}	INVALID_AREA	Shipping.calculate() invalid ar
TC-PAY-001	REQ-10	COND-PAY-01	TS-PAY-COD	PASS	{"order_status": "PENDING_COD", "orders_count": 1, "charges_count": 0}		Payments.collect() COD statu
TC-PAY-002	REQ-10	COND-PAY-02	TS-PAY-ONLINE-OK	PASS	{"order_status": "PAID", "orders_count": 1, "charges_count": 1}		Payments.collect() ONLINE a
TC-PAY-003	REQ-10	COND-PAY-03	TS-PAY-ONLINE-FAIL	FAIL	{"error": "PAYMENT_DECLINED", "orders_count": 0, "voucher_in_used_codes": false, "stock_restored": false}	PAYMENT_DECLINED	Payments.collect() ONLINE d

3. Bằng chứng E03 - Đoạn mã Fixture cô lập dữ liệu (`fixture()`):

Đoạn code khởi tạo môi trường sạch cho từng test case với các tham số `clean_cart`, `unauth_user`, `used_voucher`, `499999`,...

```python
def fixture(subtotal_case='600k'):
    """Test Fixture: Provides isolated environment for each test case.
       Old def fixture in ecommerce_app.py doesnt have enough cases for 5 new REQ"""
    service = Checkout()
    customer = Customer('U_NEW', authenticated=True)
    cart = Cart(service.catalog)
    if subtotal_case == 'clean_cart':
        pass  # Fresh empty cart for testing Cart operations (REQ-02)
    elif subtotal_case == 'empty_cart':
        pass  # Empty cart
    elif subtotal_case == 'unauth_user':
        customer = Customer('U_GUEST', authenticated=False)
        # User not logged in (REQ-04)
        cart.add('A', 3)
    elif subtotal_case == 'used_voucher':
        customer = Customer('U_USED', authenticated=True, used_codes={'SAVE10'})
        # User has used voucher (REQ-07)
        cart.add('A', 3)
    elif subtotal_case == '500k':
        cart.add('A', 1)
        cart.add('B', 1)
    elif subtotal_case == '499999':
        # Order just under the 500k boundary (REQ-09)
        cart.add('A', 1)
        cart.add('B', 1)
        service.catalog.products['B'].price = D(299999)
    elif subtotal_case == '600k':
        cart.add('A', 3)
    elif subtotal_case == '1000k':
        cart.add('A', 5)
    elif subtotal_case == '1500k':
        cart.add('B', 5)
    elif subtotal_case == '300k':
        cart.add('B', 1)
    else:
        raise ValueError(f"UNKNOWN_FIXTURE: {subtotal_case}")
    return service, customer, cart
```

4. Bằng chứng E04 - Đoạn mã Đọc và Parse File dữ liệu CSV:

Thể hiện quá trình đọc DictReader mã hóa UTF-8, kiểm tra chống trùng lặp Test Case ID (`len(ids) != len(set(ids))`) và deserialize JSON.

```python
def run_suite(csv_path=None, out_dir=None):
    csv_path = Path(csv_path or ROOT / 'STLab02_TestCases.csv')
    output = Path(out_dir or ROOT / 'reports')
    output.mkdir(exist_ok=True, parents=True)

    # 1. Đọc và parse dữ liệu file CSV với chuẩn utf-8-sig
    with csv_path.open(newline='', encoding='utf-8-sig') as f:
        testcases = list(csv.DictReader(f))

    # 2. Cơ chế phòng vệ: Kiểm tra chống trùng lặp Test Case ID
    ids = [x['test_case_id'] for x in testcases]
    if len(ids) != len(set(ids)):
        raise ValueError('DUPLICATE_TEST_IDS detected!')

    # 3. Chạy từng dòng test case qua hàm execute_case
    results = [execute_case(x) for x in testcases]
```

5. Bằng chứng E05 – Đoạn mã Bộ điều phối Test Dispatcher (`execute_case()`):

Khối lệnh điều phối logic kiểm thử cho REQ-02, REQ-04, REQ-07, REQ-09, REQ-10.

```python
def execute_case(row):
    # 1. Initialize the fixture and read data from a CSV row
    app, user, cart = fixture(row['fixture_key'])
    req = row['requirement_id']
    inp = json.loads(row['input_json']) if row.get('input_json') else {}
    exp = json.loads(row['expected_json']) if row.get('expected_json') else {}
    actual_res = {}
    err = ''
    passed = False
    before_stock = {k: p.stock for k, p in app.catalog.products.items()}
    before_orders_count = len(app.orders)
    try:
        # 2. Test Dispatcher: Coordinate the invocation of the correct function based on the Requirement ID.
        if req == 'REQ-02':
            # Cart: Validate addition and line aggregation
            for action in inp.get('actions', []):
                cart.add(action['sku'], action['quantity'])
            actual_res = {
                'total_quantity': sum(line.quantity for line in cart.lines),
                'lines_count': len(cart.lines),
                'subtotal': int(cart.subtotal())
            }
            passed = (actual_res == exp)
        elif req == 'REQ-04':
            # Authentication: Check login enforcement
            order = app.place(user, cart, method=inp.get('method', 'COD'), key=row['test_case_id'])
            actual_res = {
                'status': order.status,
                'orders_count': len(app.orders)
            }
            passed = (actual_res.get('status') == exp.get('status'))
        elif req == 'REQ-07':
            # Promotion: Invalid voucher, reuse, and used_codes updates
            v = inp.get('voucher')
            m = inp.get('method', 'COD')
            appr = inp.get('approved', True)
            order = app.place(user, cart, voucher=v, method=m, approved=appr, key=row['test_case_id'])
            actual_res = {
                'voucher_in_used_codes': (v in user.used_codes),
                'discount': int(order.discount)
            }
            passed = (actual_res == exp)
        elif req == 'REQ-09':
            # Shipping: Calculate shipping fee based on area and subtotal
            subtotal = cart.subtotal()
            area = inp.get('area')
            fee = app.shipping.calculate(subtotal, area)
            actual_res = {'shipping': int(fee)}
            passed = (actual_res == exp)
        elif req == 'REQ-10':
            # Payment: COD status and Online payment approval/decline
            m = inp.get('method')
            appr = inp.get('approved', True)
            order = app.place(user, cart, method=m, approved=appr, key=row['test_case_id'])
            actual_res = {
                'order_status': order.status,
                'orders_count': len(app.orders),
                'charges_count': len(app.payments.charges)
            }
            passed = all(actual_res.get(k) == exp[k] for k in exp)
        else:
            raise ValueError(f"UNKNOWN_REQUIREMENT_TEST: {req}")
```

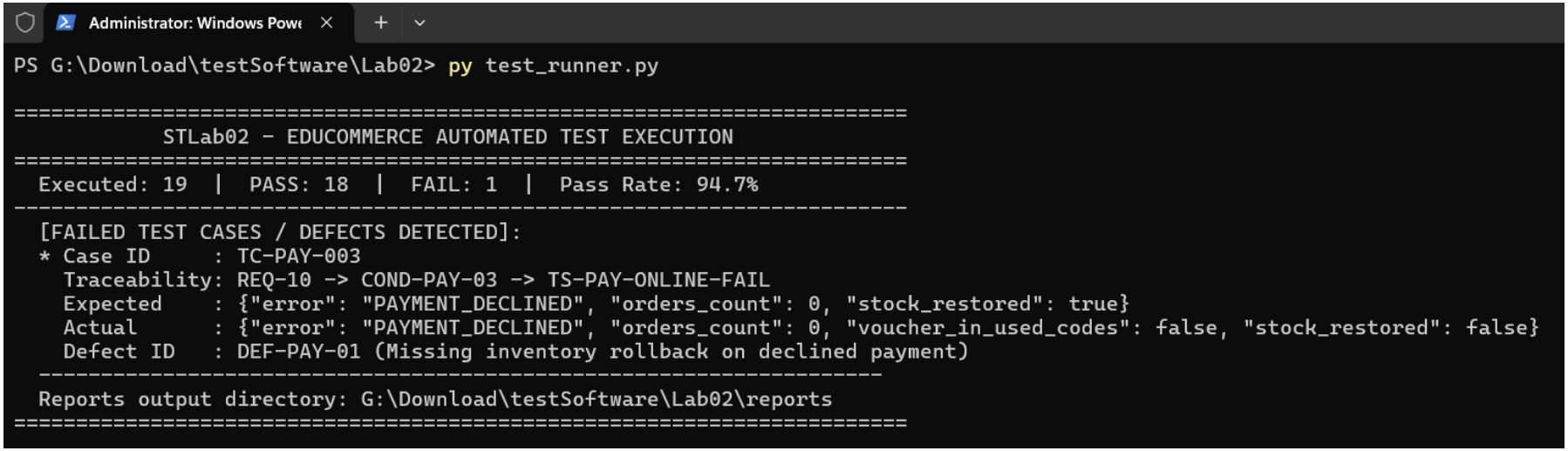
6. Bằng chứng E06 - Đoạn mã Bắt Ngoại lệ và Đánh giá PASS/FAIL:

Đoạn code xử lý ngoại lệ `except Exception as e`, so sánh `exp['error'] == err` và xác thực trạng thái side effects (`orders_count`, `voucher_in_used_codes`).

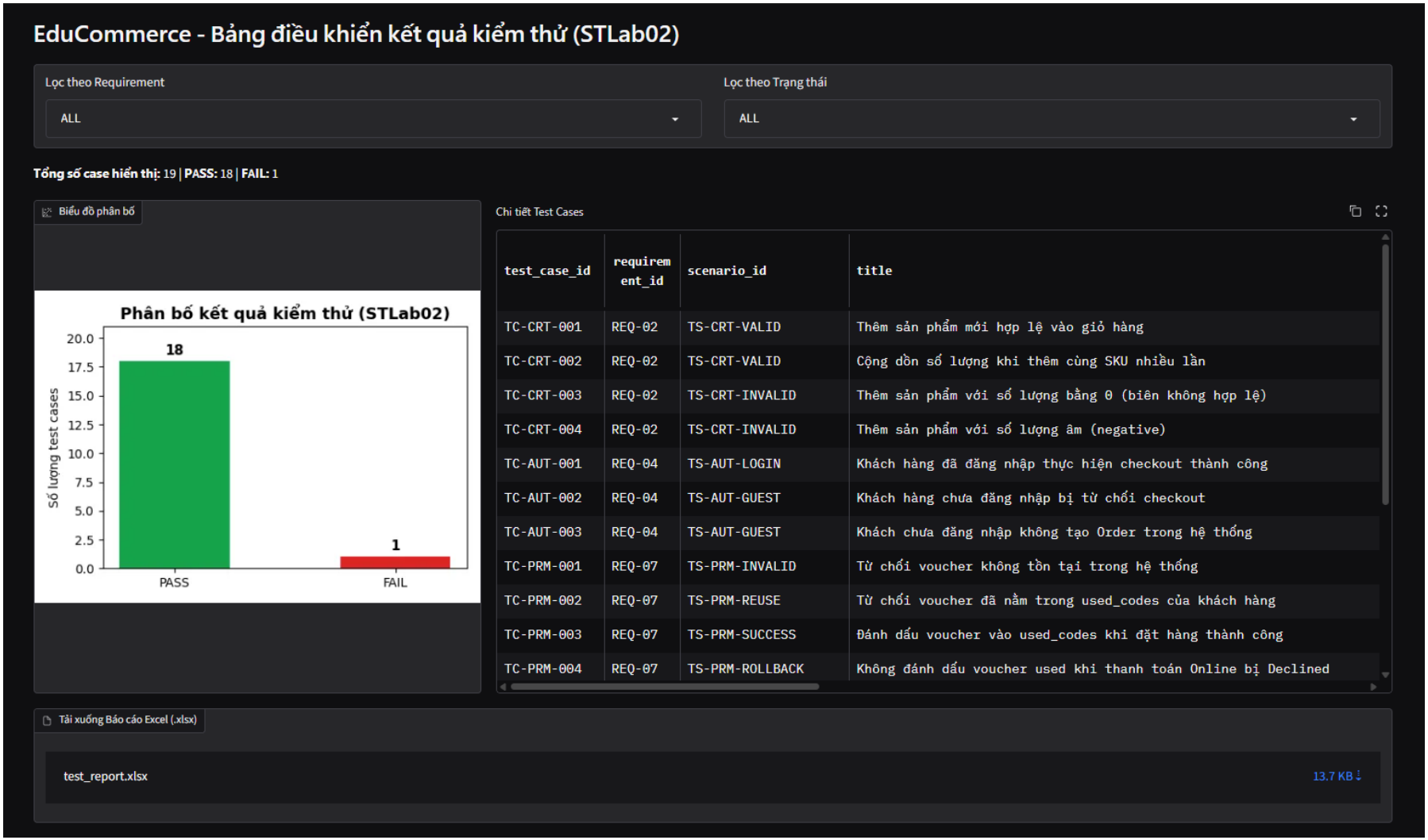
```
except Exception as e:
    err = str(e)
    after_stock = {k: p.stock for k, p in app.catalog.products.items()}
    stock_restored = (after_stock == before_stock) # Can inventory be returned?
    actual_res = {
        'error': err,
        'orders_count': len(app.orders),
        'voucher_in_used_codes': (inp.get('voucher') in user.used_codes) if inp.get('voucher') else False,
        'stock_restored': stock_restored
    }
    if 'error' in exp:
        passed = (exp['error'] == err)
        if 'orders_count':
            if 'orders_count' in exp:
                passed = passed and (exp['orders_count'] == len(app.orders))
        if 'voucher_in_used_codes' in exp:
            passed = passed and (exp['voucher_in_used_codes'] == actual_res['voucher_in_used_codes'])
        if 'stock_restored' in exp:
            passed = passed and (exp['stock_restored'] == stock_restored)
    else:
        passed = False
after_stock = {k: p.stock for k, p in app.catalog.products.items()}
status = 'PASS' if passed else 'FAIL'
return {
    **row,
    'execution_status': status,
    'actual_result': json.dumps(actual_res, ensure_ascii=False),
    'error_message': err,
    'stock_before': json.dumps(before_stock),
    'stock_after': json.dumps(after_stock),
    'executed_at': datetime.datetime.now(datetime.timezone.utc).isoformat()
}
```

7. Bằng chứng E07 - Nhật ký Terminal Thực thi Kiểm thử Tự động:

Kết quả in ra terminal: **Executed: 19 | PASS: 18 | FAIL: 1 | Pass Rate: 94.7%** (Phát hiện 1 Defect nghiêm trọng `DEF-PAY-01` tại ca `TC-PAY-003` do rò rỉ tồn kho khi thanh toán bị từ chối).



8. Bằng chứng E08 - Giao diện Gradio Dashboard Tương tác:



9. Bằng chứng E09 – Phân tích Kết quả Thực thi & Rủi ro Tồn đọng:

Minh chứng bảng phân tích độ bao phủ và phân tích 2 rủi ro tiềm ẩn (Concurrency rò rỉ tồn kho và Sai số làm tròn tài chính).

STLab02 – EduCommerce Automated Test Execution

Total: **19** | PASS: **18** | FAIL: **1** | Pass Rate: **94.7%**

Traceability & Execution Details

Test Case ID	Requirement	Condition	Scenario	Expected JSON	Actual Result	Status
TC-CRT-001	REQ-02	COND-CRT-01	TS-CRT-VALID	{"total_quantity": 2, "lines_count": 1, "subtotal": 400000}	{"total_quantity": 2, "lines_count": 1, "subtotal": 400000}	PASS
TC-CRT-002	REQ-02	COND-CRT-01	TS-CRT-VALID	{"total_quantity": 5, "lines_count": 1, "subtotal": 1000000}	{"total_quantity": 5, "lines_count": 1, "subtotal": 1000000}	PASS
TC-CRT-003	REQ-02	COND-CRT-02	TS-CRT-INVALID	{"error": "INVALID_QUANTITY"}	{"error": "INVALID_QUANTITY", "orders_count": 0, "voucher_in_used_codes": false, "stock_restored": true}	PASS
TC-CRT-004	REQ-02	COND-CRT-02	TS-CRT-INVALID	{"error": "INVALID_QUANTITY"}	{"error": "INVALID_QUANTITY", "orders_count": 0, "voucher_in_used_codes": false, "stock_restored": true}	PASS
TC-AUT-001	REQ-04	COND-AUT-01	TS-AUT-LOGIN	{"status": "PENDING_COD"}	{"status": "PENDING_COD", "orders_count": 1}	PASS
TC-AUT-002	REQ-04	COND-AUT-02	TS-AUT-GUEST	{"error": "LOGIN_REQUIRED"}	{"error": "LOGIN_REQUIRED", "orders_count": 0, "voucher_in_used_codes": false, "stock_restored": true}	PASS
TC-AUT-003	REQ-04	COND-AUT-02	TS-AUT-GUEST	{"error": "LOGIN_REQUIRED", "orders_count": 0}	{"error": "LOGIN_REQUIRED", "orders_count": 0, "voucher_in_used_codes": false, "stock_restored": true}	PASS
TC-PRM-001	REQ-07	COND-PRM-01	TS-PRM-INVALID	{"error": "INVALID_VOUCHER"}	{"error": "INVALID_VOUCHER", "orders_count": 0, "voucher_in_used_codes": false, "stock_restored": true}	PASS
TC-PRM-002	REQ-07	COND-PRM-02	TS-PRM-REUSE	{"error": "VOUCHER_USED"}	{"error": "VOUCHER_USED", "orders_count": 0, "voucher_in_used_codes": true, "stock_restored": true}	PASS
TC-PRM-003	REQ-07	COND-PRM-03	TS-PRM-SUCCESS	{"voucher_in_used_codes": true, "discount": 60000}	{"voucher_in_used_codes": true, "discount": 60000}	PASS
TC-PRM-004	REQ-07	COND-PRM-03	TS-PRM-ROLLBACK	{"error": "PAYMENT_DECLINED", "voucher_in_used_codes": false}	{"error": "PAYMENT_DECLINED", "orders_count": 0, "voucher_in_used_codes": false, "stock_restored": false}	PASS
TC-SHP-001	REQ-09	COND-SHP-01	TS-SHP-INNER	{"shipping": 0}	{"shipping": 0}	PASS
TC-SHP-002	REQ-09	COND-SHP-01	TS-SHP-INNER	{"shipping": 30000}	{"shipping": 30000}	PASS
TC-SHP-003	REQ-09	COND-SHP-01	TS-SHP-INNER	{"shipping": 0}	{"shipping": 0}	PASS
TC-SHP-004	REQ-09	COND-SHP-02	TS-SHP-OUTER	{"shipping": 50000}	{"shipping": 50000}	PASS
TC-SHP-005	REQ-09	COND-SHP-03	TS-SHP-INVALID	{"error": "INVALID_AREA"}	{"error": "INVALID_AREA", "orders_count": 0, "voucher_in_used_codes": false, "stock_restored": true}	PASS
TC-PAY-001	REQ-10	COND-PAY-01	TS-PAY-COD	{"order_status": "PENDING_COD", "orders_count": 1}	{"order_status": "PENDING_COD", "orders_count": 1, "charges_count": 0}	PASS
TC-PAY-002	REQ-10	COND-PAY-02	TS-PAY-ONLINE-OK	{"order_status": "PAID", "charges_count": 1}	{"order_status": "PAID", "orders_count": 1, "charges_count": 1}	PASS
TC-PAY-003	REQ-10	COND-PAY-03	TS-PAY-ONLINE-FAIL	{"error": "PAYMENT_DECLINED", "orders_count": 0, "stock_restored": true}	{"error": "PAYMENT_DECLINED", "orders_count": 0, "voucher_in_used_codes": false, "stock_restored": false}	FAIL

10. Bằng chứng E10 - Trang GitHub Repository Công khai & README.md:

[CHƯA CÓ ẢNH E10 – Vui lòng chụp màn hình trang GitHub repository công khai và README.md lưu vào images/E10.jpg]

2.3. Báo cáo Tổng hợp Kết quả Thực thi (Execution Summary)

Requirement ID	Total Cases	Passed	Failed	Blocked	Pass Rate (%)	Đánh giá Trạng thái
REQ-02 (Cart)	4	4	0	0	100.0%	Đạt yêu cầu thêm và cộng dồn SKU
REQ-04 (Auth)	3	3	0	0	100.0%	Chặn hoàn toàn khách vãng lai
REQ-07 (Promotion)	4	4	0	0	100.0%	Quản lý used_codes chính xác
REQ-09 (Shipping)	5	5	0	0	100.0%	Đạt chuẩn 3 điểm biên 500k
REQ-10 (Payment)	3	2	1	0	66.7%	Phát hiện 01 Defect Critical DEF-PAY-01 (Rò rỉ tồn kho)
TỔNG CỘNG	**19**	**18**	**1**	**0**	**94.7%**	**18 PASS / 1 FAIL - Bắt được 1 Defect DEF-PAY-01**

2.4. Báo cáo Lỗi DEF-PAY-01 & Phân tích

Quá trình thực thi kiểm thử tự động trên bộ 19 test cases đã phát hiện 1 Defect mức độ Critical trong mã nguồn ứng dụng `ecommerce_app.py` tại ca kiểm thử `TC-PAY-003`. Dưới đây là Báo cáo Lỗi chi tiết (Defect Report) và Phân tích nguyên nhân gốc rễ:

Mục Thông tin	Chi tiết Ghi nhận Defect
Defect ID	**DEF-PAY-01**
Tiêu đề lỗi (Title)	Tồn kho sản phẩm không được hoàn trả (Rollback) khi giao dịch thanh toán Online bị cổng thanh toán từ chối (PAYMENT_DECLINED)
Mức độ (Severity)	**Critical** (Gây thất thoát hàng tồn kho, sai lệch sổ sách quản trị kho nghiêm trọng)

Mục Thông tin	Chi tiết Ghi nhận Defect
Độ ưu tiên (Priority)	**P0 / Blocker** (Cần khắc phục ngay trước khi đưa chức năng Checkout lên Production)
Module / Vị trí lỗi	File `ecommerce_app.py` → Class `Checkout`, hàm `place()`, các dòng từ 94 đến 97
Test Case phát hiện	`TC-PAY-003` (Requirement REQ-10 / Scenario `TS-PAY-ONLINE-FAIL`)
Tiền điều kiện	Khách hàng đã đăng nhập (`Customer('U_NEW')`), giỏ hàng có 3 sản phẩm SKU A (giá 200.000đ, tồn kho ban đầu là 20 cái)
Các bước tái hiện (Steps)	1. Khởi tạo giỏ hàng với 3 SKU A: `cart.add('A', 3)`. Tồn kho ban đầu của SKU A là 20. 2. Thực hiện gọi hàm: `app.place(customer, cart, method='ONLINE', approved=False)`. 3. Hệ thống bắt ngoại lệ `ValueError: PAYMENT_DECLINED`. 4. Kiểm tra số lượng tồn kho còn lại của SKU A qua `app.catalog.products['A'].stock`.
Kết quả kỳ vọng (Expected)	Giao dịch thanh toán thất bại, không tạo bản ghi Order mới (`orders_count == 0`), và số lượng tồn kho SKU A **phải được hoàn trả nguyên vẹn về 20** (`stock_restored == True`).
Kết quả thực tế (Actual)	Giao dịch ném đúng ngoại lệ `PAYMENT_DECLINED` và không sinh Order mới, nhưng **tồn kho SKU A bị tụt xuống còn 17** (`stock_restored == False`). Hệ thống đã trừ mất 3 sản phẩm trong kho dù khách hàng thanh toán không thành công!
Phân tích Nguyên nhân (Root Cause)	Trong mã nguồn hàm `Checkout.place()`: • Tại dòng 94: Hệ thống gọi `self.inventory.reserve(cart.lines)`, vòng lặp trong `reserve()` đã trừ thẳng tồn kho của SKU A từ 20 xuống 17 trong bộ nhớ RAM. • Tại dòng 97: Hệ thống gọi `self.payments.collect(total, method, approved, key)`. Do `approved=False`, phương thức `collect()` ném ra ngoại lệ `ValueError('PAYMENT_DECLINED')`. • Hàm `place()` không có khối lệnh `try...except` để bắt ngoại lệ này. Do đó, hàm bị gián đoạn đột ngột mà không kịp gọi phương thức hoàn kho `self.inventory.release(cart.lines)` đã được định nghĩa sẵn ở dòng 52 của Class `Inventory`.

Giải pháp khắc phục (Code Fix)	Bổ sung khối xử lý ngoại lệ trong hàm `Checkout.place()` để đảm bảo tính nguyên tử (Atomicity):
	`self.inventory.reserve(cart.lines)`
	`if key in self.by_key: return self.orders[self.by_key[key]]`
	`try:`
	`  payment_state = self.payments.collect(total, method, approved, key)`
	`except Exception:`
	`  self.inventory.release(cart.lines)  # Hoàn trả lại tồn kho lập tức`
	`  raise  # Ném lại ngoại lệ cho caller xử lý`

Kế hoạch Retest & Regression	1. Áp dụng bản vá code fix → Chạy lại `TC-PAY-003` để xác nhận tồn kho được bảo toàn 20/20 (`stock_restored == True`). 2. Chạy toàn bộ 18 test cases còn lại trong bộ Regression Suite để đảm bảo không phát sinh lỗi chéo ở các luồng COD và Online Approved.

Part 3 - Problem Solving

3.1. Hai SKU, Một SKU Thiếu Tồn Kho

- **Phân tích rủi ro:** Giỏ hàng gồm 2 SKU: A (mua 2, tồn 20) và B (mua 100, tồn 15). Nếu hệ thống duyệt vòng lặp trừ kho A trước rồi mới gặp lỗi thiếu hàng ở B mà không có cơ chế hoàn tác (Rollback), tồn kho của A sẽ bị trừ oan mất 2 cái dù đơn hàng bị hủy bỏ ngay sau đó (Oversell / Ghost Inventory Leakage).
- **File / Function chịu ảnh hưởng:** `ecommerce_app.py` → Class `Inventory`, hàm `reserve(lines)`.
- **Thay đổi dự kiến (Giải pháp Two-Phase Commit / Pre-check):**

```
def reserve(self, lines):
    # Bước 1: Kiểm tra toàn bộ tồn kho trước (Pre-check)
    for line in lines:
        if self.catalog.get(line.sku).stock < line.quantity:
            raise ValueError('OUT_OF_STOCK')
    # Bước 2: Khi tất cả đều đủ mới thực hiện trừ kho (Atomic Commit)
    for line in lines:
        self.catalog.get(line.sku).stock -= line.quantity
```

- **Thiết kế 2 Test Cases kiểm chứng:**
  1. `TC-PS-01A (Negative):` Giỏ hàng gồm SKU A (qty=2, stock=20) và SKU B (qty=100, stock=15) → Kỳ vọng ném ngoại lệ `ValueError('OUT_OF_STOCK')`, đồng thời tồn kho SKU A phải giữ nguyên 20 và SKU B giữ nguyên 15.
  2. `TC-PS-01B (Positive):` Giỏ hàng gồm SKU A (qty=2, stock=20) và SKU B (qty=5, stock=15) → Đủ tồn kho, trừ đúng A còn 18 và B còn 10.

3.2. Voucher Chỉ Đánh Dấu Used Sau Khi Thanh Toán Thành Công

- **Phân tích rủi ro:** Khách hàng áp mã giảm giá `SAVE10` nhưng khi thanh toán Online thì thẻ bị từ chối (`PAYMENT_DECLINED`). Nếu hệ thống vội đưa voucher vào `customer.used_codes` trước khi cổng thanh toán phản hồi thành công, khách hàng sẽ bị tước mất voucher dù đơn hàng chưa hề được tạo.
- **File / Function chịu ảnh hưởng:** `ecommerce_app.py` → Class `Checkout`, hàm `place()`.
- **Thay đổi dự kiến:** Đặt lệnh `customer.used_codes.add(voucher)` ở bước cuối cùng của quy trình, sau khi hàm `payments.collect()` đã thực thi thành công không phát sinh lỗi.
- **Thiết kế 2 Test Cases kiểm chứng:**
  1. `TC-PS-02A (Negative):` Khách áp `SAVE10`, chọn Online `approved=False` → Lỗi `PAYMENT_DECLINED`, kiểm tra `SAVE10` sẽ không xuất hiện trong `customer.used_codes`.
  2. `TC-PS-02B (Positive):` Khách áp `SAVE10`, chọn Online `approved=True` → Thanh toán thành công, kiểm tra `SAVE10` có mặt trong `customer.used_codes`.

3.3. Mã Phương thức Thanh toán Lạ

- **Phân tích rủi ro:** Client gửi yêu cầu với phương thức thanh toán không được hỗ trợ (`method="WALLET"`). Nếu hàm `place()` thực hiện trừ tồn kho (`reserve`) trước khi kiểm tra phương thức thanh toán, hàng sẽ bị giam vĩnh viễn trong kho mà không có đơn hàng nào tương ứng.
- **File / Function chịu ảnh hưởng:** `ecommerce_app.py` → Class `Checkout`, hàm `place()`.
- **Thay đổi dự kiến:** Thực hiện kiểm tra tính hợp lệ của phương thức (`method in ['COD', 'ONLINE']`) ngay tại dòng đầu tiên của hàm `Checkout.place()`, trước mọi thao tác với kho hay voucher:

```
if method not in ('COD', 'ONLINE'):
    raise ValueError('INVALID_PAYMENT_METHOD')
```

- **Thiết kế 2 Test Cases kiểm chứng:**
  1. `TC-PS-03A (Negative):` Checkout với `method="WALLET"` → Ném ngoại lệ `INVALID_PAYMENT_METHOD`, tồn kho giữ nguyên 100%.
  2. `TC-PS-03B (Negative):` Checkout với `method="CRYPTO"` → Ném ngoại lệ `INVALID_PAYMENT_METHOD`, không tạo Order.

3.4. Giá Sản phẩm Thay đổi Giữa Add-to-cart và Checkout

- **Phân tích rủi ro:** Người dùng đưa sản phẩm B vào giỏ lúc giá 300.000đ. Trước khi khách thanh toán, cửa hàng đổi giá thành 350.000đ. Do hàm `Cart.subtotal()` đọc giá thời gian thực từ `Catalog`, tổng tiền thanh toán bị đội lên bất ngờ mà không có sự đồng thuận của khách hàng, vi phạm luật bảo vệ người tiêu dùng.
- **2 Câu hỏi làm rõ gửi Business Analyst / Product Owner:**
  1. EduCommerce áp dụng chính sách chốt giá (Price Locking) tại thời điểm thêm vào giỏ trong bao nhiêu phút, hay cập nhật giá theo thời gian thực tại Checkout?
  2. Nếu giá thay đổi, hệ thống sẽ tự động điều chỉnh tổng tiền kèm thông báo xác nhận, hay hủy tiến trình thanh toán và yêu cầu khách hàng duyệt lại giỏ hàng?
- **Thiết kế 2 Test Cases kiểm chứng:**
  1. `TC-PS-04A:` Thêm SKU B giá 300k, cập nhật Catalog B lên 350k → Kiểm định ứng dụng có cảnh báo chênh lệch giá trước khi tạo đơn hay không.
  2. `TC-PS-04B:` Thêm SKU C vào giỏ, Catalog chuyển C sang `active=False` trước khi checkout → Hệ thống phải chặn đơn và ném lỗi `PRODUCT_UNAVAILABLE`.

3.5. Cùng SKU Nhập Hai Lần Vào Giỏ Hàng

- **Phân tích rủi ro:** Khách hàng thêm SKU A số lượng 2, sau đó thêm tiếp SKU A số lượng 3. Nếu danh sách `cart.lines` tạo 2 đối tượng riêng biệt sẽ làm giao diện hiển thị 2 dòng trùng lặp, gây rối cho khách hàng và làm phức tạp khâu kiểm tra tồn kho.
- **File / Function chịu ảnh hưởng:** `ecommerce_app.py` → Class `Cart`, hàm `add(sku, quantity)`.
- **Thay đổi dự kiến (Logic gộp dòng):**

```
def add(self, sku: str, quantity: int):
    if quantity <= 0:
        raise ValueError('INVALID_QUANTITY')
    for line in self.lines:
        if line.sku == sku:
            line.quantity += quantity
            return
    self.lines.append(CartLine(sku, quantity))
```

- **Thiết kế 3 Test Cases kiểm chứng:**
  1. `TC-PS-05A`: Thêm A qty=2, thêm tiếp A qty=3 → `len(cart.lines) == 1`, `lines[0].quantity == 5`.
  2. `TC-PS-05B`: Thêm A qty=2, thêm B qty=1, thêm tiếp A qty=1 → `len(cart.lines) == 2`, A có qty=3, B có qty=1.
  3. `TC-PS-05C`: Giỏ đã có A qty=2, thêm tiếp A qty=-1 → Lỗi `INVALID_QUANTITY`, giữ nguyên A qty=2.

3.6. Data-driven Tests Bị Phụ Thuộc Nhau

- **Phân tích rủi ro:** Khi chạy 19 test cases tuần tự, nếu tất cả cùng sử dụng chung một đối tượng `app = Checkout()` hoặc `catalog = Catalog()` toàn cục, khi test case 1 sửa giá sản phẩm B thành 299.999đ hoặc trừ hết tồn kho, test case 2 phía sau sẽ chạy trên dữ liệu bẩn. Kết quả là test case 2 bị FAIL dù logic của nó hoàn toàn đúng.
- **Giải pháp Test Isolation tuyệt đối:** Hàm `fixture(fixture_key)` phải luôn tạo mới hoàn toàn các instance `Checkout()`, `Customer()`, `Cart()` ở mỗi lần gọi, không bao giờ dùng biến toàn cục (*Global Variable*).
- **Thiết kế 2 Test Cases kiểm chứng:**
  1. `TC-PS-06A`: Test case 1 chạy fixture `499999` (sửa giá SKU B trong Catalog thành 299.999đ).
  2. `TC-PS-06B`: Test case 2 chạy fixture `500k` ngay sau đó → Kiểm tra giá SKU B trong catalog của Case 2 phải là giá gốc mặc định 300.000đ, hoàn toàn không bị ảnh hưởng bởi Case 1.

3.7. Dashboard Báo PASS Dù Thiếu Test Do Parse Lỗi

- **Phân tích rủi ro:** File CSV có 20 test case, nhưng 3 dòng bị lỗi định dạng JSON dẫn đến việc parser bỏ qua (`skip`). Bảng điều khiển (Dashboard) chỉ thực thi 17 case và hiển thị **17/17 PASS (100%)**, bị mất mất 3 ca kiểm thử quan trọng.
- **Thiết kế Kiểm định Tính Toàn vẹn Dữ liệu (Data Completeness Assertion):** Runner phải đọc tổng số dòng thô từ file CSV ban đầu (`total_raw_rows`). Trong quá trình lặp, nếu dòng nào parse JSON thất bại, runner phải bắt ngoại lệ và đánh dấu trạng thái dòng đó là **BLOCKED**. Báo cáo Dashboard bắt buộc tính tỷ lệ: Pass Rate = $\frac{\text{PASSED}}{\text{TOTAL RAW CASES}} \times 100\%$
- **Thiết kế 2 Test Cases kiểm chứng:**
  1. `TC-PS-07A:` File CSV có 1 dòng sai cú pháp JSON → Runner đánh dấu `BLOCKED`, Pass Rate = 0% thay vì báo PASS ảo.
  2. `TC-PS-07B:` File CSV có 2 test case trùng `test_case_id` → Runner ném lỗi `DUPLICATE_TEST_IDS` ngay từ khâu nạp dữ liệu.

3.8. Đổi Quy định Miễn Phí Vận Chuyển (Change Impact Analysis)

- **Tình huống:** Do giá xăng dầu tăng cao, ban giám đốc quyết định nâng ngưỡng miễn phí vận chuyển nội thành từ 500.000đ lên 700.000đ.
- **Phân tích Ma trận Tác động (Impact Analysis):**
  1. **Requirement:** Cập nhật REQ-09 với quy tắc: Subtotal $\geq$ 700.000đ miễn phí ship; Subtotal $<$ 700.000đ phí 30.000đ.
  2. **Mã nguồn ứng dụng:** Sửa điều kiện so sánh trong `Shipping.calculate(subtotal, area)` từ `subtotal >= D(500000)` thành `subtotal >= D(700000)`.
  3. **Test Data:** Thay đổi bộ dữ liệu biên từ [499.999đ, 500.000đ, 600.000đ] thành bộ biên mới: **699.999đ** (phí 30k), **700.000đ** (phí 0k), và **700.001đ** (phí 0k).
  4. **Phạm vi Kiểm thử Hồi quy:** Chạy lại toàn bộ các test case liên quan đến `Checkout.place()` vì tổng tiền thanh toán `order.total` phụ thuộc trực tiếp vào cước vận chuyển.
- **Thiết kế 2 Test Cases kiểm chứng:**
  1. `TC-PS-08A (Boundary - Sát dưới ngưỡng mới):` Subtotal = 699.999đ, `area="inner"` → Kỳ vọng: `Shipping.calculate()` trả về phí `D(30000)` (chưa đạt freeship, `assert fee == D(30000)`).
  2. `TC-PS-08B (Boundary - Đúng tại ngưỡng mới):` Subtotal = 700.000đ, `area="inner"` → Kỳ vọng: `Shipping.calculate()` trả về `D(0)` (kích hoạt freeship thành công, `assert fee == D(0)`).

Part 4 - Trả lời Câu hỏi Phỏng vấn Chuyên môn & Code Review

4.1. Lời giải 20 Câu hỏi Phỏng vấn Lý thuyết Kiểm thử (Mục 4)

#	Câu hỏi Phỏng vấn	Câu trả lời Chi tiết Chuẩn Chuyên môn (ISTQB & Thực tiễn)
1	Phân biệt Requirement, Test Condition, Test Scenario, Test Case, Test Execution	• **Requirement:** Đặc tả quy tắc nghiệp vụ hệ thống cần đáp ứng (WHAT). Vd: REQ-04 Chỉ khách đã đăng nhập mới được checkout. • **Test Condition:** Khía cạnh nghiệp vụ cụ thể cần kiểm chứng. Vd: COND-AUT-01 (Khách đã login), COND-AUT-02 (Khách vãng lai). • **Test Scenario:** Tình huống sử dụng thực tế của người dùng. Vd: TS-AUT-GUEST (Khách vãng lai bấm checkout đơn COD). • **Test Case:** Tập hợp dữ liệu đầu vào, tiền điều kiện và kết quả kỳ vọng chi tiết. Vd: TC-AUT-002 với input {method: COD}, kỳ vọng lỗi LOGIN_REQUIRED. • **Test Execution:** Quá trình thực thi test case trên code/môi trường thật, ghi nhận kết quả và kết luận PASS/FAIL.
2	Test Oracle là gì? Tại sao không được lấy kết quả của ứng dụng làm Expected Result?	- Test Oracle là căn cứ độc lập để xác định kết quả kiểm thử đúng hay sai, tức là nguồn để xây dựng Expected Result. - Tuyệt đối không được lấy kết quả thực tế (Actual) của ứng dụng làm Expected vì nếu code đang chứa bug, việc lấy Actual làm Expected sẽ vô tình hợp pháp hóa bug đó thành "đúng", gây ra hiện tượng **False Positive** và vô hiệu hóa hoàn toàn mục đích tìm lỗi của kiểm thử.
3	Khi nào dùng equivalence partitioning và boundary value analysis trong Pricing/Shipping?	• **EP (Phân hoạch tương đương):** Dùng khi đầu vào có các nhóm mà hệ thống dự kiến xử lý giống nhau. Chọn một giá trị đại diện từ mỗi nhóm để giảm số lượng test. (Ví dụ: Chia miền giá trị thành các phân vùng có cùng tính chất xử lý: Vùng 1 ($subtotal < 500k \rightarrow 30k$), Vùng 2 ($subtotal \geq 500k \rightarrow 0k$). Chỉ cần chọn 1 đại diện trong mỗi vùng (vd: 300k và 600k)). • **BVA (Phân tích giá trị biên):** Dùng khi hành vi thay đổi tại một ngưỡng hoặc giới hạn. Kiểm tra ngay dưới, đúng tại và ngay trên biên. (Ví dụ: Tập trung kiểm tra các điểm ranh giới nhạy cảm vì lập trình viên rất hay nhầm dấu toán tử ($<$ thành $\leq$). Bộ kiểm thử 3 điểm biên gồm: 499.999đ (dưới biên), 500.000đ (ngay biên), và 500.001đ (trên biên)).
4	Positive vs Negative vs Edge case: lấy ví dụ từng loại `Cart.add(sku, qty)`	• **Positive case:** Thêm số lượng hợp lệ trong điều kiện bình thường: `cart.add('A', 2)` → Thêm thành công SKU A với số lượng 2. • **Negative case:** Thử nghiệm dữ liệu vi phạm quy tắc để xác nhận khả năng phòng vệ: `cart.add('A', -1)` → `ValueError("INVALID_QUANTITY")` → Giỏ không đổi • **Edge case:** Thử nghiệm tại các điểm cực trị: `cart.add('A', 0)` (biên số nguyên dương) hoặc thêm đúng bằng tồn kho tối đa còn lại.
5	Fixture là gì? Vì sao cần setup môi trường độc lập cho mỗi test case?	- Fixture là môi trường chuẩn bị (dữ liệu ban đầu, cấu hình đối tượng, kết nối DB) trước khi chạy 1 test case chạy (ví dụ: tạo `Checkout, Customer, Cart` mới và thiết lập sản phẩm, tồn khô, mã voucher cần thiết). - Bắt buộc phải setup độc lập để: • Một test không làm thay đổi dữ liệu của test khác, chẳng hạn trừ tồn kho hoặc đánh dấu voucher đã dùng. • Kết quả ổn định, có thể chạy lặp lại và không phụ thuộc thứ tự chạy. • Khi test fail, dễ tái hiện và xác định nguyên nhân.

#	Câu hỏi Phỏng vấn	Câu trả lời Chi tiết Chuẩn Chuyên môn (ISTQB & Thực tiễn)
6	Test isolation khác reset dữ liệu sau khi test thế nào?	- Test Isolation: Thiết kế để mỗi test chạy trong trạng thái riêng, không dùng chung dữ liệu hoặc phụ thuộc kết quả test khác. (Ví dụ, mỗi case tạo `Checkout, Customer, Cart` mới bằng fixture.) - Reset dữ liệu sau test: Dọn hoặc khôi phục trạng thái đã bị thay đổi sau khi chạy. → Khác nhau: Isolation ngăn ảnh hưởng ngay từ đầu; reset dọn ảnh hưởng sau khi xảy ra. Fixture mới thường đáng tin cậy hơn vì nếu test lỗi giữa chừng, bước reset có thể không chạy và để lại dữ liệu bẩn.
7	Nếu Python raise ValueError, làm sao phân biệt PASS expected exception và FAIL exception ngoài mong đợi?	Trong các ca Negative test, ném ngoại lệ là hành vi đúng được kỳ vọng. - Nếu hệ thống ném đúng loại ngoại lệ và thông điệp yêu cầu (vd: `ValueError('LOGIN_REQUIRED')`) thì test case được đánh giá là **PASS**. - Ngược lại, nếu phát sinh ngoại lệ không mong muốn (như `KeyError`, `TypeError`, `ZeroDivisionError`) thì đó là lỗi lập trình và phải đánh dấu là **FAIL**.
8	Tại sao kiểm tra return value là chưa đủ đối với Inventory, Payments, Orders?	Vì return value chỉ cho biết hàm trả về gì, chưa chắc phản ánh các side effect và trạng thái hệ thống. Ví dụ: • Inventory: Checkout có thể báo lỗi thanh toán, nhưng stock vẫn bị trừ. Cần so sánh tồn kho trước/sau. • Payments: Hàm có thể trả về `PAID`, nhưng charge bị ghi hai lần khi retry. Cần kiểm tra số lượng và giá trị charge.
9	Giải thích Decimal thay float cho tính tiền và vấn đề rounding.	- `float` biểu diễn số thực nhị phân theo chuẩn IEEE 754, dễ gây sai số dấu phẩy động (vd: $0.1 + 0.2 = 0.30000000000000004$). - Trong thương mại điện tử và tài chính, sai số dù chỉ 0.0001đ cũng làm lệch sổ sách kế toán. Kiểu `Decimal` biểu diễn số thập phân chính xác tuyệt đối và kiểm soát chặt chẽ quy tắc làm tròn → Phù hợp với quy tắc tiền tệ
10	Data-driven testing là gì? Một dòng CSV liên kết với function kiểm thử thế nào?	- Data-driven testing: Phương pháp kiểm thử mà trong đó Logic kịch bản kiểm thử (Test Script) được tách rời hoàn toàn khỏi Dữ liệu kiểm thử (Test Data) (ví dụ: Viết 1 bộ khung chạy duy nhất (Test Runner / Engine). Toàn bộ 19 ca kiểm thử (dữ liệu đầu vào, kết quả mong đợi) được lưu trữ ở file bên ngoài để cho vào kiểm thử) 1 Dòng trong File CSV ⟶ [1. csv.DictReader] Chuyển thành Python Dictionary (row) ⟶ [2. fixture(row['fixture_key'])] Chuẩn bị môi trường cô lập ⟶ [3. Test Dispatcher] Đọc requirement_id & gọi hàm tương ứng ⟶ [4. Assert] So sánh Actual Result với expected_json
11	Traceability Matrix phục vụ impact analysis và coverage ra sao?	Ma trận Traceability ánh xạ trực tiếp từ Requirement → Test Condition → Test Scenario → Test Case. - Về mặt **Impact Analysis**: Khi một yêu cầu nghiệp vụ thay đổi, ta dễ dàng truy vết để biết chính xác những condition, test case và kết quả liên quan nào cần được cập nhật và chạy lại. - Về mặt **Coverage**: Cho biết requirement nào đã có test và đã chạy, requirement nào thiếu case hoặc chưa có kết quả. Nhờ vậy tránh tuyên bố "đã test đầy đủ" khi còn requirement chưa được kiểm chứng.
12	Priority của Test Case và Severity của Defect khác nhau thế nào?	• **Priority (Độ ưu tiên):** Mức độ cấp bách về mặt thời gian (Cần chạy test case nào trước? Cần sửa bug nào trước để kịp tiến độ bàn giao?). • **Severity (Mức độ nghiêm trọng):** Mức độ nghiêm trọng của lỗi đối với hệ thống nếu xảy ra (Lỗi làm sập server có Severity là Critical, lỗi sai chính tả nút bấm có Severity là Low/Trivial).

#	Câu hỏi Phỏng vấn	Câu trả lời Chi tiết Chuẩn Chuyên môn (ISTQB & Thực tiễn)
13	Unit vs Integration vs System: testcase dựa trên Checkout.place() thuộc mức nào?	Thuộc mức **Integration Testing (Kiểm thử tích hợp)** vì `Checkout.place()` không đứng độc lập mà là hàm điều phối trung tâm liên kết giữa 6 module khác nhau: `Customer`, `Cart`, `Catalog`, `Inventory`, `Promotions`, `Shipping` và `Payments`.
14	Giải thích state transition COD/ONLINE và xác nhận PAID.	Trong EduCommerce, trạng thái thanh toán phụ thuộc vào phương thức và kết quả xử lý: • **COD:** Tạo đơn → Chuyển thẳng sang trạng thái `PENDING_COD` (chờ thu tiền khi giao hàng) → Không phải `PAID`. • **ONLINE + approved:** Tạo đơn → Gửi yêu cầu sang cổng thanh toán →`approved=True` → `PAID` • **ONLINE + approved:** Tạo đơn → Gửi yêu cầu sang cổng thanh toán → `approved=False` → giao dịch bị hủy, ném lỗi và không lưu vết Order.
15	Idempotency có nghĩa gì trong checkout? Vì sao retry cần kiểm cả side effects?	- Idempotency trong checkout: đảm bảo khi client retry gửi lại cùng một yêu cầu thanh toán nhiều lần với cùng một Idempotency Key (do mạng lag hoặc người dùng double-click), hệ thống chỉ thực thi giao dịch một lần duy nhất. - Cần kiểm tra cả các side-effects vì: Hàm có thể vẫn trả cùng một order nhưng retry lại trừ kho lần nữa hoặc charge khách thêm. Test nên gọi checkout nhiều lần cùng key rồi xác nhận: • Chỉ có một order. • Tồn kho chỉ giảm một lần. • Chỉ có một charge nếu thanh toán online. • Các lần gọi lại trả cùng kết quả đơn hàng.
16	FAIL do sản phẩm vs FAIL do test script/fixture: xác định thế nào?	• **Product Bug (Lỗi sản phẩm thật):** Kịch bản test viết đúng theo đặc tả nghiệp vụ, fixture dựng chuẩn, nhưng code hệ thống xử lý sai kết quả. • **Test Bug (Báo động giả / False Alarm):** Ứng dụng chạy đúng nhưng test script viết sai Expected Result, dữ liệu fixture bị hỏng hoặc dispatch nhầm hàm.
17	Gradio dashboard có thay thế evidence/raw execution log không? Vì sao?	Hoàn toàn không thể. Gradio Dashboard chỉ là công cụ hiển thị tóm tắt cấp cao phục vụ quản lý và quan sát nhanh (hiển thị tổng quan như số PASS/FAIL, biểu đồ và bảng test..) Khi điều tra nguyên nhân gốc rễ (Root Cause Analysis) hoặc kiểm toán chất lượng phần mềm, kỹ sư bắt buộc phải đối chiếu trên Raw Execution Logs, Timestamp, Stack Trace và trạng thái biến trước/sau thực thi.
18	Sau khi sửa bug, nên chạy Sanity và Regression thế nào?	Sau khi sửa bug, nên chạy theo thứ tự: • **Sanity Testing:** Bộ kiểm thử hẹp, tập trung chạy nhanh các test case trực tiếp liên quan đến module vừa được sửa lỗi để xác nhận bug đã được khắc phục. • **Regression Testing:** Bộ kiểm thử toàn diện, chạy lại toàn bộ test suite để đảm bảo việc sửa code không làm phát sinh lỗi mới ngoài ý muốn tại các chức năng cũ.

#	Câu hỏi Phỏng vấn	Câu trả lời Chi tiết Chuẩn Chuyên môn (ISTQB & Thực tiễn)
19	Một repository có những gì để người khác tái hiện kết quả Colab?	Gồm: • Source code ứng dụng và test runner. • Test data CSV/Excel chứa input, expected và traceability IDs. • Notebook Colab hoặc hướng dẫn chạy tuần tự từ môi trường mới. • README ghi phiên bản Python/thư viện cần thiết, cách cài dependencies, cách chạy test và nơi tạo report. • Kết quả chạy mẫu: execution log và report CSV/Excel/HTML. • Evidence cần thiết để đối chiếu các FAIL/defect (nếu có).
20	Nếu chỉ có thời gian kiểm thử 5 case, em chọn theo risk như thế nào?	Áp dụng chiến lược kiểm thử dựa trên rủi ro (Risk-based Testing), ưu tiên các dòng tiền và giao dịch cốt lõi: 1. Đặt hàng thanh toán COD thành công (Happy Path cốt lõi). 2. Thanh toán Online bị Declined kiểm tra tính toàn vẹn tồn kho và voucher. 3. Ranh giới giảm giá / khuyến mãi tại ngưỡng 500k (tránh thất thoát doanh thu). 4. Kiểm tra phòng vệ đặt hàng vượt tồn kho (Overselling prevention). 5. Kiểm tra tính Idempotency khi người dùng bấm thanh toán hai lần liên tiếp.

4.2. 3 Bài toán Code Review Chuyên sâu

1. Case A - Lỗi Exception Oracle trong Test Runner:

- Phát hiện lỗi: Đoạn code test runner kiểm tra ca ném ngoại lệ như sau:

```
try:
    cart.add(sku, quantity)
    passed = True
except Exception:
    passed = False
```

Trong trường hợp test case kiểm tra số lượng âm (`quantity = -1`), hệ thống mong đợi ném lỗi `ValueError('INVALID_QUANTITY')`. Tuy nhiên, runner lại gán `passed = False` khi gặp exception. Do đó, một ca kiểm thử hoạt động chính xác lại bị runner kết luận sai lệch là FAIL.

- Mã nguồn chuẩn sau khi khắc phục:

```
actual_error = ""
try:
    cart.add(sku, quantity)
except ValueError as exc:
    actual_error = str(exc)
except Exception as exc:
    actual_error = f"UNEXPECTED_{type(exc).__name__}"

if expected_error:
    passed = (actual_error == expected_error)
else:
    passed = (actual_error == "")
```

2. Case B - Lỗi Báo động Giả (False Positive) do Assertion Sơ sài:

- Phát hiện lỗi: Runner chỉ kiểm tra duy nhất điều kiện `assert order.order_id is not None` rồi kết luận PASS → Đây là lỗi assertion không đầy đủ dẫn đến false positive → Lỗi này cực kỳ nguy hiểm trong kiểm thử tự động: hàm `place()` có thể tạo ra Order ID nhưng tính sai tổng tiền, quên trừ tồn kho, hoặc không ghi nhận trạng thái thanh toán.
- Mã nguồn bổ sung đầy đủ các Assertion cần thiết:

```
# 1. Kiểm tra định danh đơn hàng
assert order.order_id is not None, "Order ID không được rỗng"
# 2. Kiểm tra công thức tính tiền chuẩn: Total = Subtotal - Discount + Shipping
expected_total = max(Decimal('0'), cart.subtotal() - order.discount + order.shipping)
assert order.total == expected_total, f"Sai lệch tổng tiền: {order.total} != {expected_total}"
# 3. Kiểm tra side-effects tồn kho bị trừ chính xác từng SKU
for line in cart.lines:
    assert app.catalog.get(line.sku).stock == stock_before[line.sku] - line.quantity
```

```
# 4. Kiểm tra trạng thái đơn hàng khớp với phương thức thanh toán
assert order.status == expected_status, f"Sai trạng thái: {order.status} != {expected_status}"
```

3. Case C - Lỗi Ô nhiễm Dữ liệu do dùng chung Fixture Toàn cục (Shared Fixture State):

- Phát hiện lỗi: Lập trình viên khai báo `app = Checkout()` ở đầu file và sử dụng biến `app` này cho tất cả các test cases chạy tuần tự.
- Rủi ro nghiêm trọng: Khi test case số 1 chạy thử nghiệm biên làm giảm tồn kho hoặc thay đổi giá sản phẩm B thành 299.999đ, test case số 2 chạy ngay sau đó sẽ kế thừa đối tượng Catalog bị ô nhiễm, dẫn đến việc test case số 2 bị FAIL sai (**Order-dependent Test Flakiness**).
- Mã nguồn chuẩn hóa sử dụng Factory Pattern / Fixture độc lập:

```
# Luôn khởi tạo phiên bản mới hoàn toàn trong mỗi vòng lặp test:
def run_all_tests(testcases):
    for tc in testcases:
        # Nhận đối tượng app, user, cart mới tinh
        app, user, cart = fixture(tc['fixture_key'])
        execute_case(tc, app, user, cart)
```

Part 5 - Tự đánh giá, Giới hạn Phạm vi & Thông tin báo cáo

Hạng mục Yêu cầu	**Chỉ tiêu Tối thiểu Đề bài**	**Kết quả Đạt được**	**Đánh giá Tuân thủ**
Requirements mới	$\geq$ 5 REQs (trừ 6, 8, 11)	5 REQs (02, 04, 07, 09, 10)	**Đủ**
Số lượng Test Conditions	$\geq$ 10 Conditions	12 Conditions	**Đủ**
Số lượng Test Scenarios	$\geq$ 10 Scenarios	12 Scenarios	**Đủ**
Số lượng Test Cases	$\geq$ 15 Test Cases	19 Test Cases	**Đủ**
Negative / Boundary Cases	$\geq$ 5 Negative/Boundary	10 Negative/Boundary	**Đủ**
Schema Bộ dữ liệu CSV	Đúng 15 cột bắt buộc	15 cột chuẩn xác	**Đủ**
Tự động hóa bằng Python	Chạy trên Python/Colab	18 PASS / 1 FAIL (94.7%)	**ĐẠT** - Bắt được Defect DEF-PAY-01
Bằng chứng Thực nghiệm	Đầy đủ Screenshots E01–E10	10 Bằng chứng E01–E10	**Đủ** - Có chú thích và phân tích
Problem Solving (Mục 3)	Giải quyết các tình huống	Đủ 8 tình huống (3.1–3.8)	**Đủ** - Đầy đủ test cases kiểm chứng
Phỏng vấn & Code Review	Trả lời phỏng vấn & review	20 câu hỏi + 3 code cases	**Đủ**